

Report Submission Cover sheet

Course Lecturer: Dr Clay Chu

Elective Topic: [Power Electronics](#)

Team Number and Mentor Name: ['Electrical Hazard'](#), [Wilson Lee](#) and [Anastasia Smyth](#)

Since this work counts toward your formal assessment for this course, please write your name and student ID where indicated below, and sign to acknowledge your agreement with the following declaration. Add this cover sheet at the beginning of your report (cover page of your report). You can use digital signature or manual signature and add the scan-copy of this cover sheet to the beginning of your report. Failing to fill-out this form and attach it to the front of your submission will result in marks being deducted from your report (Do not use any other cover sheet forms).

Please read and tick all the boxes that applies below. Violating your declaration will result in an **immediate zero mark** on your entire Elective Topic mark plus potential consequences.

- ☐ We declare that this assessment item is our own work, except where acknowledged, and has not been submitted for academic credit elsewhere, and we acknowledge that the assessor of this item may, for the purpose of assessing this item:
- Reproduce this assessment item and provide a copy to another member of the University; and/or,
 - Communicate a copy of this assessment item to a plagiarism checking service (where a copy of the assessment item may then be retained on its database for the purpose of future plagiarism checking).
- ☐ We certify that we have read and understood the University Rules in respect of Student Academic Misconduct and Plagiarism.
- ☐ We have returned all the items borrowed from the Workshop EE-G15 and signed the return list before submitting this report and signing this cover sheet.
- ☐ We returned our E-Cores (and ultracapacitor) to the staff in EE-G15 (Power Electronics teams only).

Student ID	Family Name	First Name	Signature
z5257418	CHEN	Natalie	
z5309677	GUNASEKERA	Shalindra	
z5310260	ANDERSON	COREY	

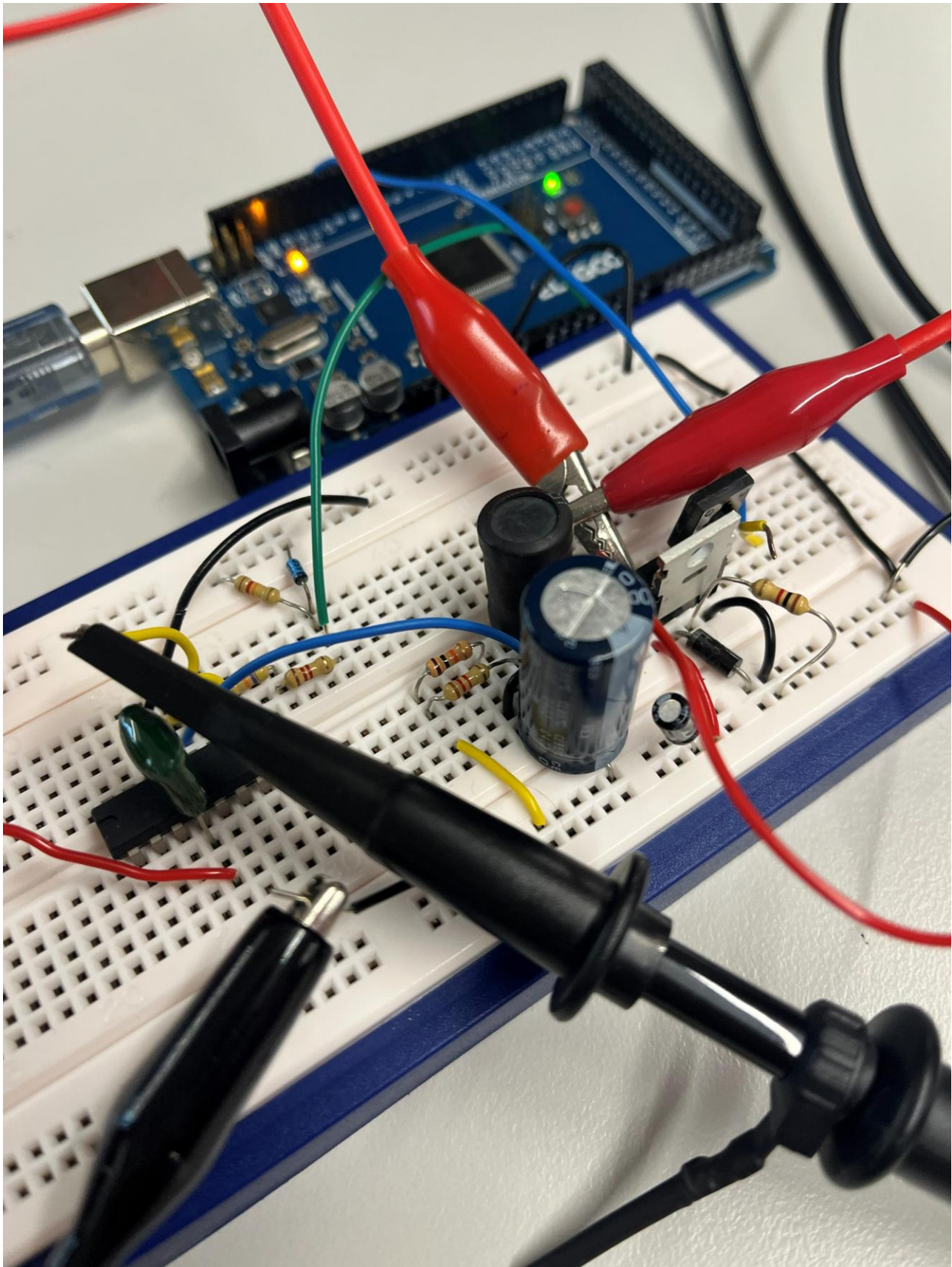


ELEC4123
Term 3, 2025

Solar System Design
By Electrical Hazard

Abstract

Two systems were created. The first was a buck converter switch-mode-power supply (SMPS) which was input-sourced from a photovoltaic (PV) panel and run using a microcontroller with a PI controller and negative feedback. This system was successful, creating a low-noise and stable 5 V output. It had reproducible and consistent operation over a load range of 100 Ω to 10 k Ω . The second was a boost converter SMPS with Maximum Power Point Tracking (MPPT) which needed further work to be able to operate from a PV panel. In future, work should be done to make the buck and boost converters operate together.



PMOS Buck converter with PI control, Electrical Hazard, iPhone 13, 2025.

Table of Contents

Introduction	6
Problem Statement	6
Task allocation	6
Tasks & Results	7
Overall topology design – C. Anderson.....	7
Simulink – C. Anderson	8
Controller Design – C. Anderson	8
Boost converter design – C. Anderson	10
MPPT design – S. Gunasekera	11
Noise reduction – S. Gunasekera	13
Digital inputs & sensing design – S. Gunasekera.....	14
Buck converter design – N. Chen	16
Converter choice	16
Component sizing	16
Buck converter topology.....	17
High-side N-channel topology (classic).....	17
Low-side N-channel topology	18
High-side P-channel topology	19
Conduction mode.....	20
Results.....	20
Software & hardware protection – N. Chen.....	22
Gate driver design – N. Chen	24
Boost converter gate driver design.....	24
Buck converter gate driver design	25
Simulink interface to Arduino design – N. Chen.....	25
Risk Mitigation Strategies.....	27
Reflection.....	28
Conclusion	29
Bibliography	30
Appendix A	31
Risk Assessment Matrix	31
Appendix B	31
Current sensor reading code	31

Introduction

Cheap, abundant, and climate positive, photovoltaics are the heavyweights in renewable energy generation and the decarbonisation of our power grids. Among the various types of solar panel generation systems are stand-alone systems – the type of system our team has been tasked into building. Specifically, a system made from off-the-shelf components, democratising the ability to harness the power of the sun.

Problem Statement

Our mission was to create a power system converter for the solar panel. This system needed to power two loads. The first load replicated a battery and had the ability, via a switch, to either output a fixed 10 V or the voltage at max power. The second load replicated a mobile phone charger, outputting a fixed 5 V over the load range of 100 Ω to 10 k Ω .

The main key challenges were noise reduction, stabilising the output when the input was variable (i.e. solar panel output), and inter-system dynamics. Noise reduction – to meet the objectives of $\Delta V = 0.3 \text{ A}$ and $\Delta I = 0.015 \text{ A}$ – was substantial due to the high-frequency of switching noise at the gates of the MOSFETs used for the SMPS and the inductance in the ground loops. Another challenge was to get the SMPS to work when the inputs were constantly changing. As the solar panel is a variable voltage source (i.e., it follows the photovoltaic “VI” curve), the buck and boost converter need to work despite that dynamism. Inter-system dynamics was also a challenge. Where the workings of buck and boost converters when the input is a stable voltage source and the gate levels are driven by a standard signal generator, are well researched, putting them together in a new configuration needed careful engineering. It also had to integrate with the Simulink control system, needing careful consideration of allowed voltage levels, control logic, compatible Arduino libraries, sampling and frequency, and analogue to digital (and vice versa) conversions.

Other challenges – albeit more specific – included: getting the buck converter to work linearly when the source gate is floating and bootstrap gate driver IC’s are not available, finding other ways to drive the MOSFET gates of the SMPS, sorting out the compatibility of Maximum Power Point Tracking Perturb and Observe algorithms with the control loop, and keeping the converters in Continuous Conduction Mode (CCM) for linear operation.

Task allocation

Table 1 describes how the main tasks were split between team members. It includes a brief description of each task’s deliverables.

Table 1 - List of key tasks and allocations to group members.

Key Task	Brief Description	Allocation
Simulink design and validation	Implementing the entire system on Simulink for validation.	C. A
Overall topology design	Layout the overall converter design and determining how converters will be connected.	C. A
Controller design	Choosing and designing an appropriate controller (PI/PD/PID).	C. A

Boost converter design	Design and verify the boost converter stage.	C. A
MPPT design	Determine a suitable maximum power point tracking (MPPT) algorithm for the solar panel and converters.	S. G
Noise reduction	Explore and implement techniques to reduce output voltage noise.	S. G
Digital input/sensing design	Design and implement circuits to measure key output signals for use in control algorithms.	S. G
Buck converter design	Design and verify the buck converter stage.	N. C
Software & hardware protection design	Implement appropriate protection to ensure the system is not exposed to dangerous or unstable signals, such as preventing over-voltage or non-physical/damaging control signal values.	N. C
Simulink-interface Arduino design	Implement a Simulink file to program and control the Arduino microcontroller.	N. C
Gate driver design	Design and build a circuit to drive the switching MOSFET in each converter.	N.C

Tasks & Results

Overall topology design – C. Anderson

The system required three functions from one PV power source: a regulated 10V across load 1, 5V across load 2 and maximum power point operation for load 1. To satisfy this, a boost-then-buck cascade topology was chosen.

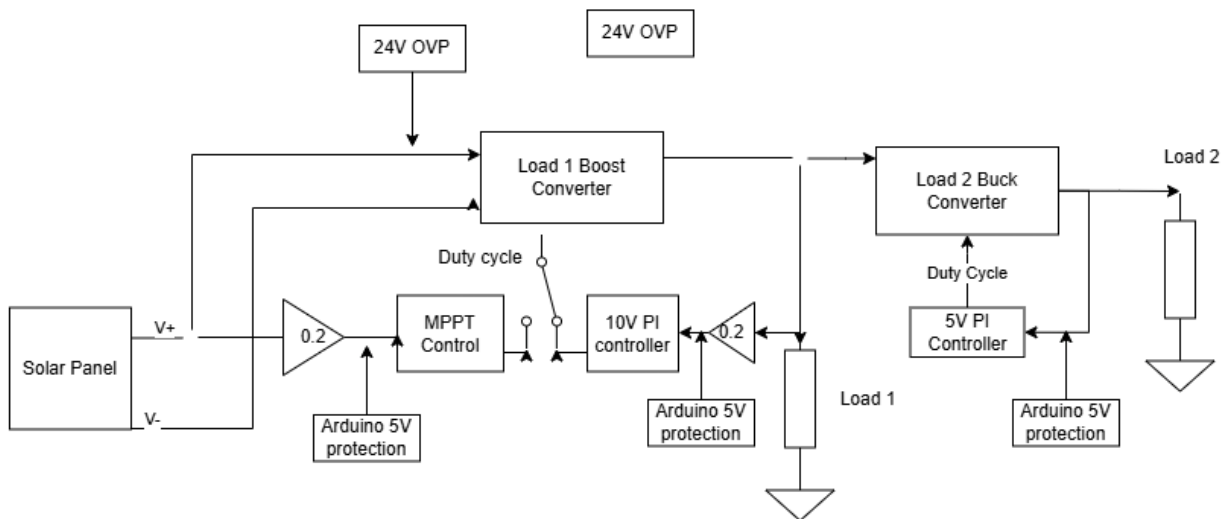


Figure 1 - High level diagram of the DC converter topology.

The output of the boost supplies the 10V for load 1 and the input of the buck. The output of the buck supplies 5V for load 2. The boost is required since the panel voltage falls below 10V when the loads are attached, especially with low irradiances. Putting the boost first also benefits the

MPPT operation since it can directly change the boost voltage without waiting for buck voltage to stabilise.

We chose cascaded topology because in parallel, both converters would load the PV panel simultaneously and compete for the operating point. A change in one duty cycle alters the panels voltage and causes the other converter's control loop to retune. In the cascade configuration, the boost first provides the 10V for load 1, the buck then regulates that 10V down to 5V for load 2. The downside of this is that the buck needs to wait for the boost to stabilise which will slow the buck reaching steady state.

Our Simulink models did not behave well for the parallel topology which confirmed our choice for the cascade topology.

Simulink – C. Anderson

The system was developed and validated in Simulink before hardware implementation. We used switches to change between the conditions (high/low irradiance) and operating modes (MPP/10V). The goal was to produce a controller structure directly linked to key challenges we identified in our problem statement: variable PV voltage, interaction between stages, noise and the risk of DCM.

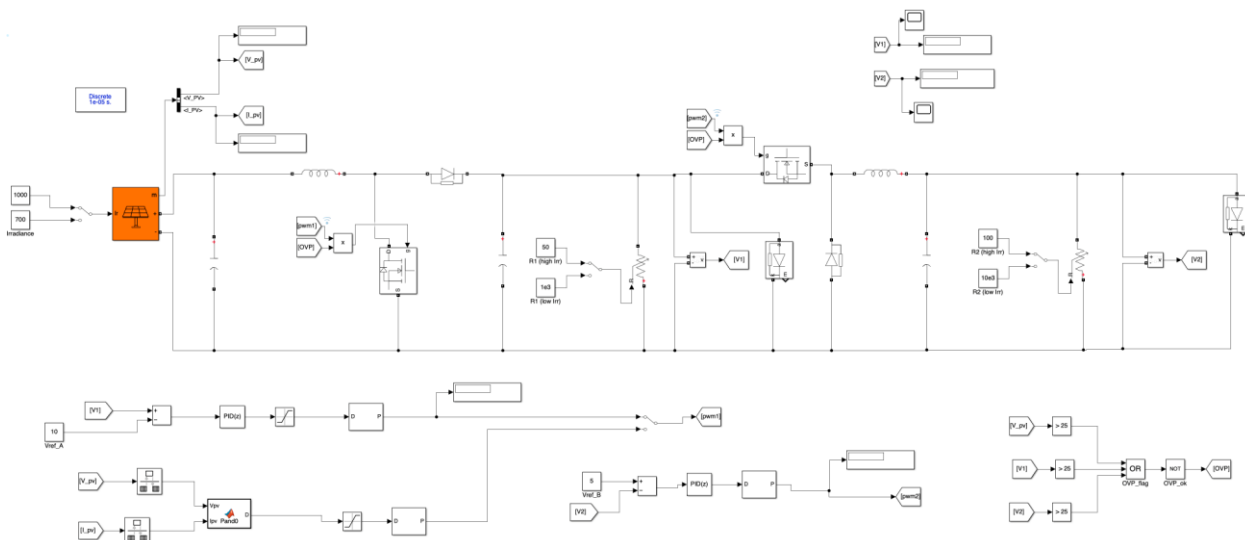


Figure 2 – Simulink file

Controller Design – C. Anderson

We used a PI controller to control the duty cycle for our converters based on the input error (difference between load voltage and reference voltage). Although a PID controller was initially tested, the system outputs did not justify us using the derivative term. The response is already well-damped by the proportional term and the integral term was required to eliminate steady state error. In simulation, the PI controller met all the design requirements. The PI controller also reduced the risk in the hardware implementation, the derivative action is highly sensitive to real-world measurement imperfections even if it behaves well in simulation. To tune the gains, we used the following method:

- 1) Set $K_I = 0$
- 2) Set K_P to a small value and increase it to speed up the response. Set the value to slightly before the point where we see clear oscillations or a big overshoot.
- 3) Add a small K_I and increase it gradually until before oscillations.
- 4) Confirm it satisfies the specifications.

We ended up with $P_I = 0.1$ and $K_I = 1$ which satisfies the specifications as seen in the plot below for the boost output at 10V and the buck at 5V.

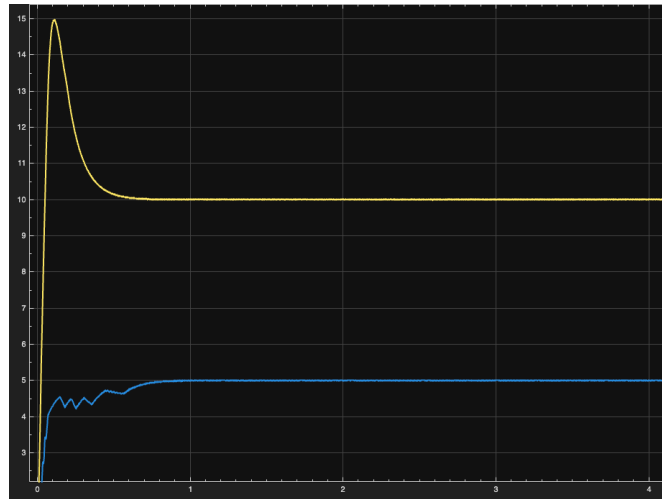


Figure 3 – The controlled boost and buck converters achieving the reference voltage. The 10V is the boost output and the 5V is the buck output.

The controller for the buck converter in the real implementation had a much longer settling time, and we did not get enough time to tune it for the response to satisfy the 4s settling time. This was due to our Simulink file being too ideal, in real systems we have non-ideal L, C and R, wiring inductance and ESR.

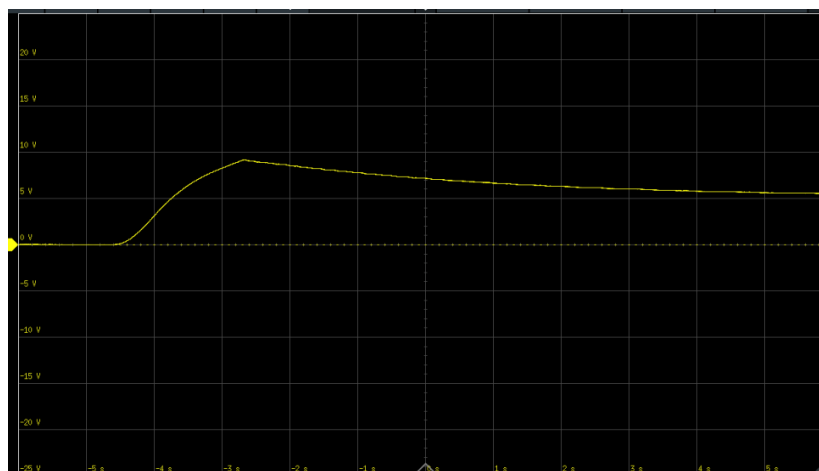


Figure 4 – Settling time for our real buck circuit

Boost converter design – C. Anderson

We used a boost converter to step up the input voltage to 10V. The open circuit voltage of the solar panel is above 10V, but with the loads attached it falls under which requires the boost, especially for low irradiances.

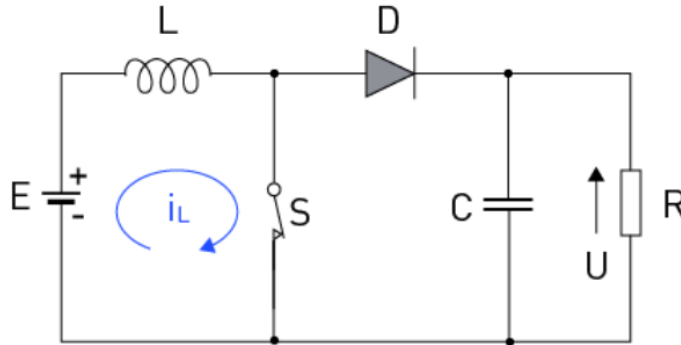


Figure 5 – Boost converter topology

The boost converter was designed from the standard continuous conduction mode relation:

$$V_{out} = \frac{V_{in}}{1-D},$$

which defines the duty range required to achieve a certain output voltage. The inductor value was chosen to avoid DCM using:

$$L = \frac{V_{in}D}{\Delta I_L f_s},$$

the ripple current had to remain below the average inductor current. CCM mode reduces the output voltage ripple and peak currents which increases efficiency. The output capacitor helps reduce voltage ripple and stabilise the output. It is calculated by the equation: [1]

$$C_{out} = \frac{I_{out}D}{\Delta V_{out}f_s},$$

using the set requirements of:

$$V_{in} = 4V \text{ (solar panel voltage with load)}$$

$$D = 1 - \frac{V_{in}}{V_{out}} = 1 - \frac{4}{10} = 0.6$$

$$\Delta I_L = 15mA \text{ (from specs)}$$

$$f_s = 30kHz \text{ (this frequency was chosen to ensure our components weren't too large)}$$

$$I_{out} = 0.1A \text{ (measured from simulation)}$$

$$\Delta V_{out} = 0.3V \text{ (from specs)}$$

This gave my inductor and capacitor values to be 5.3mH and 6.7uF, in the real implementation we used a 10mH inductor and 1mF capacitor since we found those sizes helped stabilise the output more. The actual implementation worked at first, but it somehow stopped working and our debugging attempts failed, so we do not have any photos of our boost working in this section.

MPPT design – S. Gunasekera

A perturb and observe (PO) algorithm was constructed to track the solar panel's maximum power point for a given irradiance level. PO was chosen as using a fractional open-circuit voltage method would require identification of open circuit voltages for various irradiances, which would require further modelling and result in greater error in tracking.

PO also had advantages over other algorithms such as the Incremental Conductance algorithm, which was more complicated to implement – on a microcontroller with two PI controllers, a straightforward algorithm was required. The Incremental Conductance algorithm however does settle on the maximum power point, something the PO cannot do since the algorithm must perturb off the maximum power point. Due to measurement precision however, it was believed that the Incremental Conductance algorithm would also be unable to settle on the maximum power point.

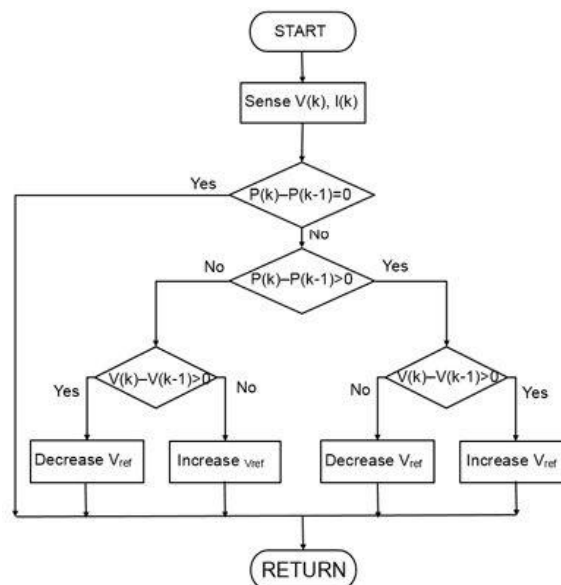


Figure 6 - Block diagram representing the Perturb and Observe algorithm. It should be noted that at every cycle, V_{ref} is perturbed even if the first condition is met, as this is the fundamental principle of the PO algorithm. Figure from .

Like other MPPT algorithms, PO measures the solar panel voltage and current, and outputs an integer representing the duty cycle that the converter switch must operate at to produce an output voltage that results in the maximum solar panel power. Changing the duty cycle achieves this, as the change results in a lower or higher input impedance – this is what allows the solar panel to change its operating point.

A high-level description of the PO method is shown above in Figure 6. First, the solar panel voltage and current are measured as the previous samples V_{k-1} and I_{k-1} . The system is then perturbed by a small change in duty cycle – this initial change can be in any direction, increase or decrease. It was found that smaller changes reduced the ‘ripple’ around the maximum power

point, so the duty cycle change was $\Delta_{duty} = 0.3\%$, corresponding to the minimum step in duty cycle on the ATMEGA 2560.

The system then measures the panel voltage and current V_k and I_k , and compares the power output of the solar panel $P_k = V_k I_k$ with the previous power P_{k-1} . If the difference was positive, so that the small increase in duty cycle resulted in an increase of solar panel power, the same duty cycle increase would be applied again. If a decrease in power was observed, the direction of duty cycle change would be reversed.

The main decision-making code is presented in Figure 7 and shows the additional layer of control to make sure the duty cycle change is going in the correct direction based on the change in solar panel voltage.

```

19 % Increase or decrease duty cycle based on conditions
20 if (Ppv-Pprev) ~= 0
21     if (Ppv-Pprev) > 0
22         if (Vpv-Vprev) > 0
23             D = Dprev - deltaD;
24         else
25             D = Dprev + deltaD;
26         end
27     else
28         if (Vpv-Vprev) > 0
29             D = Dprev + deltaD;
30         else
31             D = Dprev - deltaD;
32         end
33     end
34 else
35     D = Dprev;
36 end

```

Figure 7 - Code implementation of PO algorithm. The code used is modified from [2].

As the first boost converter was non-functional, the PO algorithm could not be verified in hardware. The performance in simulation however could be analysed and suggests the algorithm can adjust the solar panel operating point, however, has some steady state error associated with some irradiance levels. As seen in Table 2, the algorithm does well to track the maximum power point of the solar panel when at full irradiance, but has a growing error when irradiance is lowered.

Table 2- MPPT theoretical settling power compared with simulation results for high and low irradiance levels. Theoretical power levels were derived from the Simulink PV array IV characteristics display.

Irradiance ($W \cdot m^{-2}$)	Theoretical P_{max} (W)	Simulation P_{max} (W)
1000	10.2	10.1
700	7.17	5.85
50	0.48	0.43
30	0.28	0.16

A step response conducted in Simulink shows that the algorithm has low power ‘ripple’ and fast response time for a change within a high irradiance bracket. The response shown in Figure 8 had a simulated ripple of 0.15W peak to peak, and a settling time of 260ms.

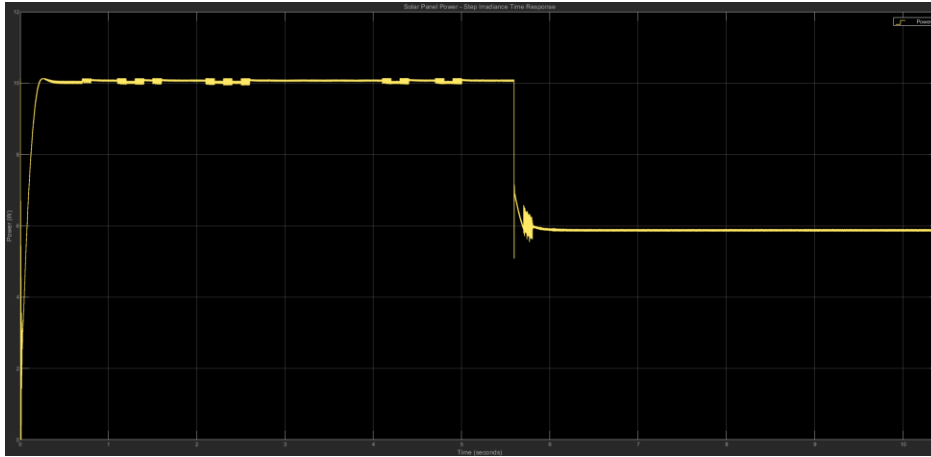
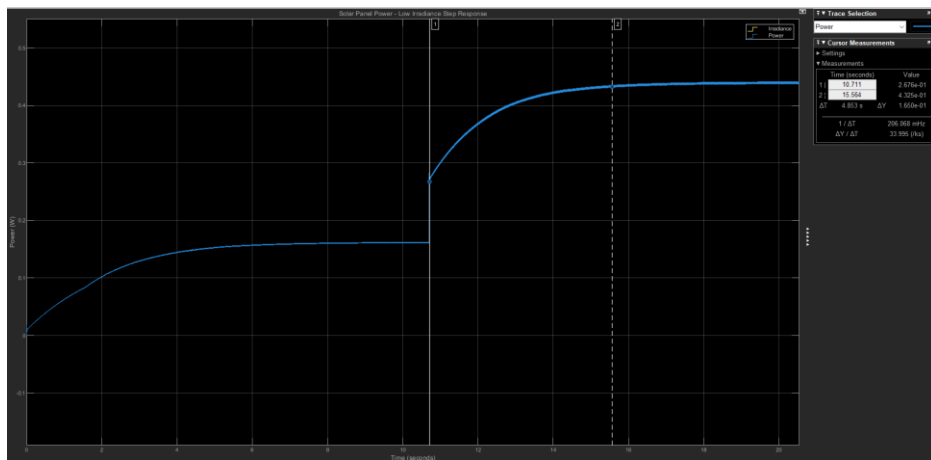


Figure 8 – Time response of MPPT algorithm for a step change in irradiance from 1000 to 700 $W \cdot m^{-2}$.

With a low irradiance step change however, the settling time increased to 4.8 seconds as shown in Figure 9. This is likely due to the small Δ_{duty} , which could only change the duty cycle by 0.3% every 0.1 seconds. The low irradiance also corresponds to lower current, the portion in which the IV characteristics have a flat gradient for current. The small Δ_{duty} therefore would take longer to adjust the system for low irradiances than higher irradiances. In hindsight, this could be improved by making Δ_{duty} dynamic, having a different value for lower irradiance levels.



ground. Despite this, the ringing persisted. Finally, the alligator clips used to make initial measurements were replaced with 10:1 measurement probes. The new ripple can be observed in Figure 10 and **Error! Reference source not found.**. Not only did this eliminate the large switching spikes, but also eliminated any trace of an underdamped ringing response. It is likely that the alligator clips either introduced significant inductance to the ground path, or the measurement points created ground loops.

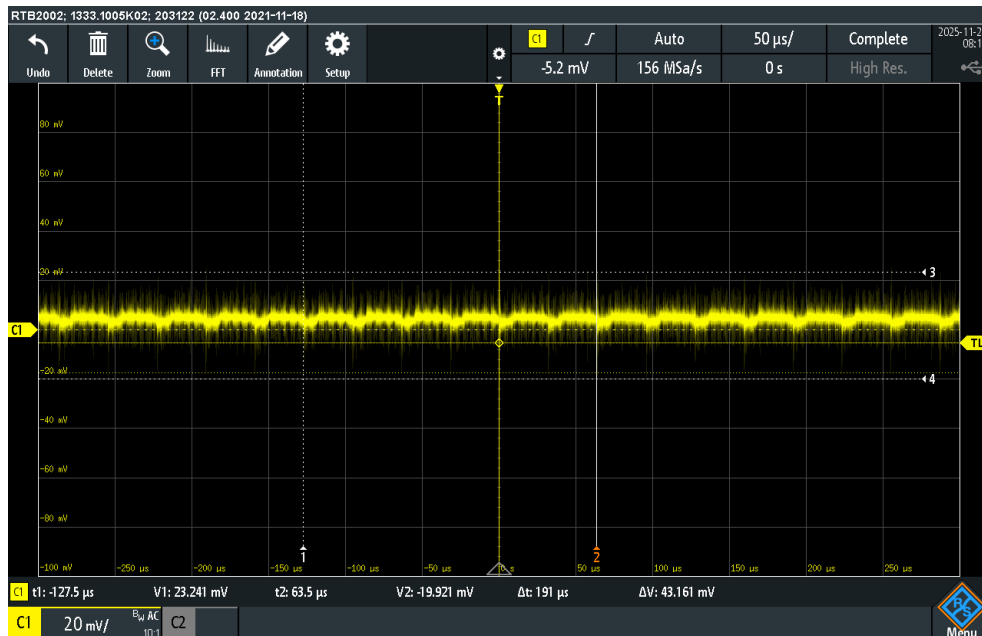


Figure 10 - Voltage ripple on 5V DC output with 1kΩ load resistor.



Figure 2 - Voltage ripple on 5V DC output with 10kΩ load resistor.

The system required real-time monitoring of three node voltages and the current drawn from the solar panel, the voltage readings requiring attenuation to then be connected to the 5V rated Arduino pins.

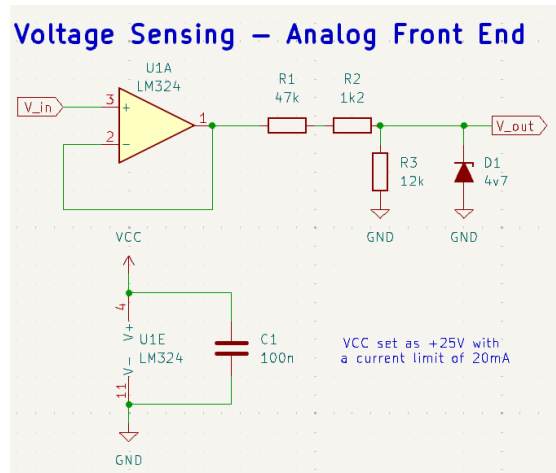


Figure 12 - Schematic of analog front end used for voltage sensing. This circuit was used for each of the three voltages measured.

For voltage sensing, each node was connected to a voltage follower cascaded with a resistor divider as shown in Figure 12. The voltage follower stage buffers the signal to ensure minimal current is drawn from the system outputs by the circuit for sensing. The LM324 operation amplifier was driven by a 25V supply and was chosen for its ability to swing close to the negative power rail, in this case ground. As each node had it's dedicated front end circuit, the circuit in **Error! Reference source not found.** was replicated three times.

The resistor divider network was designed to divide V_{in} by a factor as close to 5 as possible. This was achieved by carefully selecting resistors in the laboratory that resulted in

$$\frac{R_3}{R_1 + R_2 + R_3} \approx 5,$$

where the resistances R_1 , R_2 and R_3 correspond to the resistors in Figure 12. In the physical circuit, two of the dividers had measured ratios of 4.998 and 5.002, deemed sufficiently close to five for accurate sensing. The third divider was not measured, an oversight from the designers.

A 4.7V Zener diode was placed at the output in reverse bias to clamp the output voltage for microcontroller protection. This Zener voltage was chosen as the next highest component available had a Zener voltage of 5.6V, exceeding the ATMEGA2560 chip absolute pin voltage of 5.5V [4]. The output was then connected to three dedicated analogue pins of the Arduino board.

Current sensing was achieved by connecting the input terminals of the ACS712-5A in series with V_+ of the solar panel. The sensor converts a current measurement into voltage using a hall sensor with a nominal sensitivity $S = 185 \text{ mV} \cdot \text{A}^{-1}$. This sensitivity was not verified using a known current, again an oversight from the designers.

```

26      % --- When window full, compute new average ---
27      if count >= N
28          avgAdc = sum / N;
29          Vout = avgAdc * LSB - zeroPoint;
30          I_new = Vout / scale_factor;
31
32          % Update the held current
33          I_prev = I_new;
34
35          % Reset accumulators
36          sum = 0;
37          count = 0;
38      end

```

Figure 13 - Code used to average current sensor readings and output average as a current reading. The variable 'count' keeps track of how many samples have been added, computing the average when exceeding the window size N.

The voltage output of the sensor was connected directly to an analog pin on the Arduino, which computed the average reading using a 100-sample window. This window size was chosen to fix a 0.1 second sampling time for power measurements, as faster current measurements would introduce undesirable noise. The average was then converted from a voltage reading into the equivalent current measurement using the conversions as shown in the code snippet in Figure 13 – first the raw ADC reading is converted back into a voltage using the ADC resolution and reference voltage, and then converted into a current reading using the nominal sensitivity factor.

Buck converter design – N. Chen

Converter choice

For the requirement of getting a 5V output with loads of various resistor sizes to imitate a phone charging load, a buck converter was used which reduces the voltage from a high voltage to a lower voltage. This can be achieved through other methods such as a linear-drop-out (LDO) regulator or a simple resistor divider network however these methods are inefficient power-wise, and the voltage will drop significantly with load. On the other hand, buck converters are highly efficient, have less heat dissipation, and can handle higher currents. One drawback is the SMPS introduces switching noise and ripple. As low ripple is a requirement of the system, a section of this report has already been dedicated to it and will not be covered here.

Component sizing

The main elements in the buck converter design are as follows. The blueprint for calculations and the full proof behind the working of the buck converter is available in Xiao's *Power Electronics Step-by-Step: Design, Modelling, Simulation, and Control*. [5] First, input and output voltage, switching frequency, and current and voltage ripple were specified, in order, as:

$$\begin{aligned}
 V_{in} &= 10 \text{ V}, \\
 V_o &= 5 \text{ V}, \\
 f_{sw} &= 31372.55 \text{ Hz}, \\
 \Delta I_L &= \pm 15 \text{ mA}, \\
 \text{and } \Delta V_o &= \pm 0.3 \text{ V}.
 \end{aligned}$$

The input voltage was chosen as 10V, as this is the output of the boost converter. Switching frequency was selected around 31,372.55 Hz as a higher switching frequency was more efficient, and this was the highest frequency the Arduino Mega 2560 could output. Then, the duty ratio and 'on' time were calculated as:

$$D_{on} = \frac{V_o}{V_{in}} = 0.5,$$

$$\text{and } T_{on} = \frac{D_{on}}{f_{sw}} = \frac{0.5}{31372.55} = 15.937 \mu s.$$

From here, the minimum inductor and capacitor value could be chosen as:

$$L_{min} = \frac{V_{in} - V_o}{\Delta I_L} T_{on} = 5.31mH \rightarrow L = 10mH$$

$$\text{and } C_{o,min} = \frac{\Delta I_L}{8f_{sw}\Delta_o} = 0.199\mu F \rightarrow C_o = 1000\mu F,$$

which fit the minimum brief and helped to reduce noise as much as possible.

Buck converter topology

Multiple iterations of the buck converter were conceived. This subsection provides a brief overview of each.

High-side N-channel topology (classic)

The first iteration was the classic N-Channel buck converter as seen in Figure 14 and 15. However, this did not end up working because the output became capped at 2.5 V. This was due to the output being limited by the gate voltage plus some upper-threshold of N-Channel MOSFET. In other words, when the PWM went 'high' (5 V), the difference between the gate and source voltage allowed the MOSFET to enter its linear operating region ($V_{GS} > V_{th}$), however as soon as the output voltage rose to be close to 5 V ($V_{GS} < V_{th}$), the MOSFET dropped back into its cut-off region. Therefore, the highest output voltage would always be lower than the gate voltage.

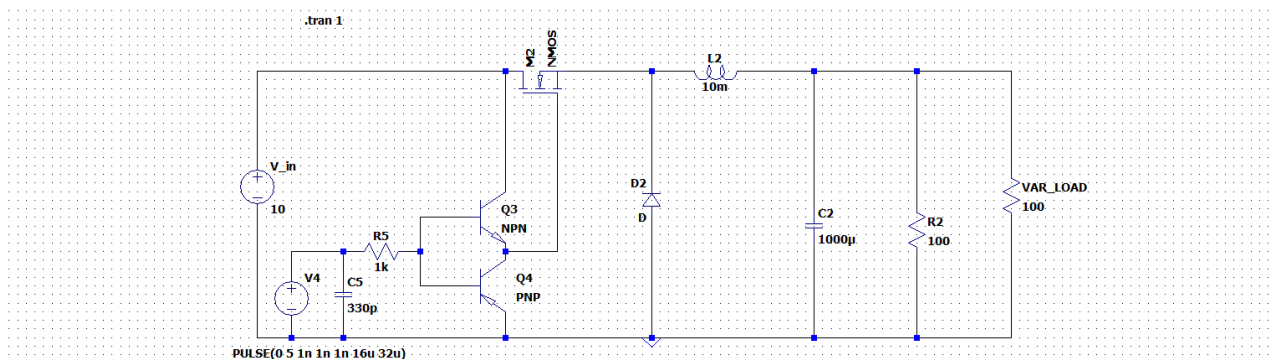


Figure 14: Classical buck converter configuration – high-side converter using NMOS.

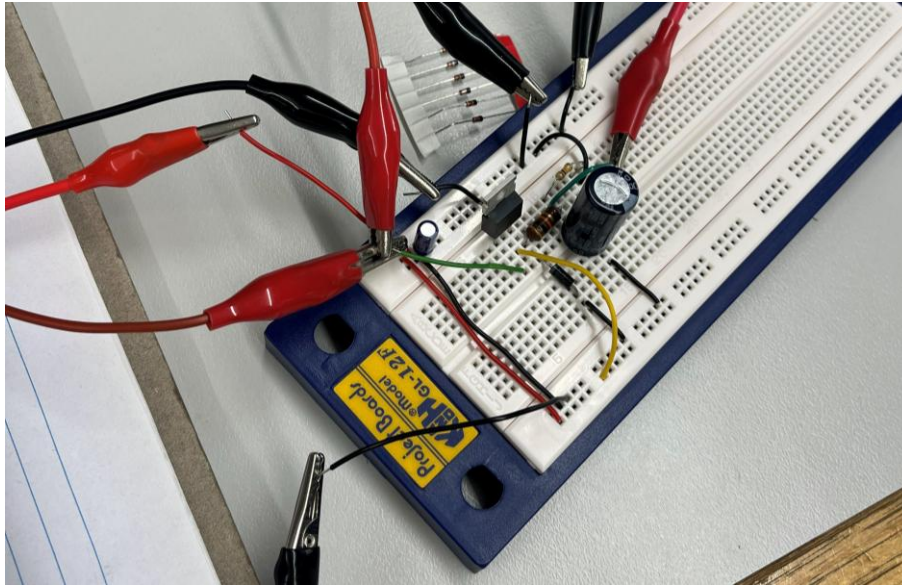


Figure 15: Classical buck converter breadboard.

Low-side N-channel topology

In the second iteration, since we had no access to a bootstrap gate driver (to boost the gate voltage) for the first iteration, a low-side buck inverter was built as seen in Figure 16 and 17. The source of the N-Channel MOSFET was now tied to ground allowing a change in duty ratio at the gate to be proportional to the output of the Buck. While this worked initially with the buck being able to produce anywhere from 2 to 8 V (with a Buck input of 10 V) using the signal generator as the PWM source, it was not able to work with the Arduino Simulink control system. This was because *increasing* the duty ratio, caused the output voltage to *decrease*, and vice versa. Since the MPPT PID control algorithm expects that the input variable (duty ratio) and the output variable (voltage) move in the same direction (and this *does* occur in the textbook buck inverter), to reverse this, breaks the system. The result was that the microcontroller tried to reduce the output voltage to get it to the reference voltage by *decreasing* the duty ratio – instead, it needed to *increase* it.

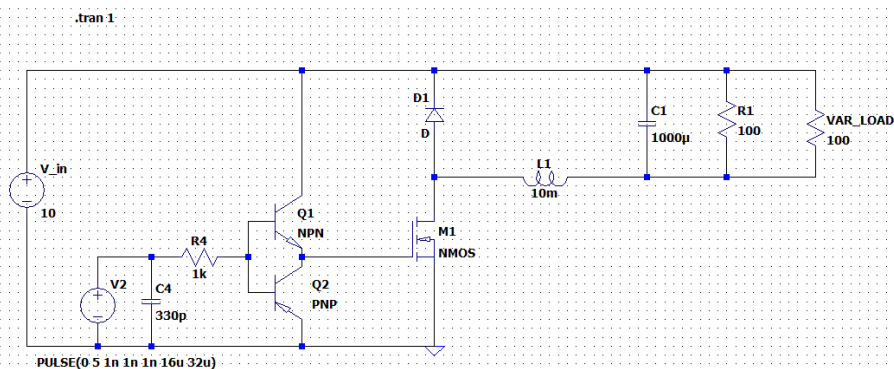


Figure 16: Low-side buck converter configuration using NMOS.

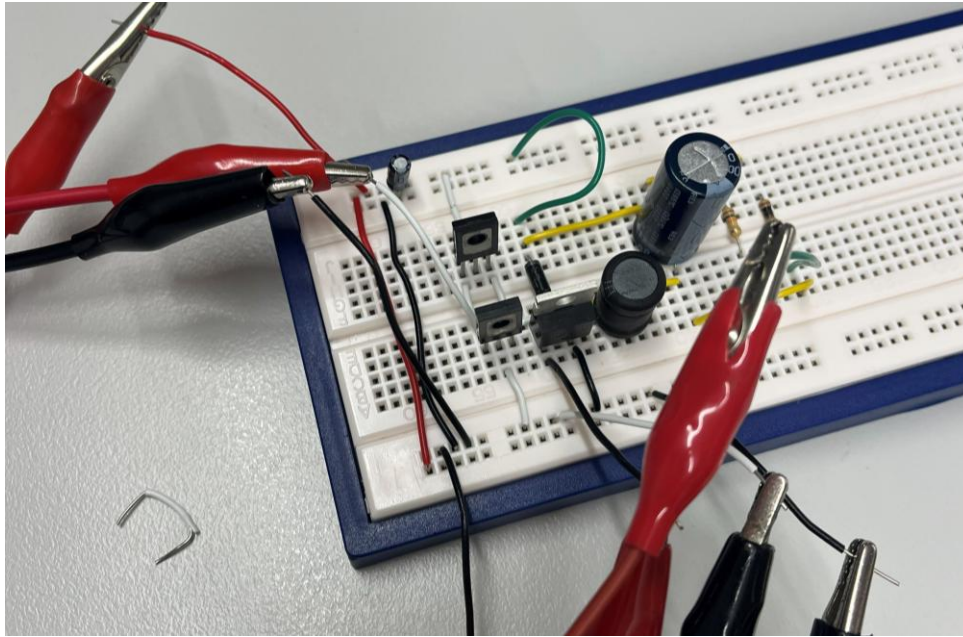


Figure 17: Low-side buck converter breadboard

High-side P-channel topology

In the third iteration, the high-side buck inverter was used again but with a P-channel MOSFET instead – see Figure 18 and 19. As opposed to the first iteration, it was much easier to pull the gate lower than the source (to 0 V) using the microcontroller than vice versa, allowing transistor switching to occur more seamlessly. It should be noted that with its advantages, PMOS has a higher drain-source resistance ($R_{DS(on)}$) creating more heat and thus is not advantageous for high-power converters. To allow faster switching and the ability for the gate voltage to be able to switch to 0 V, a simple level-shifter was devised using an NPN transistor and a resistor network.

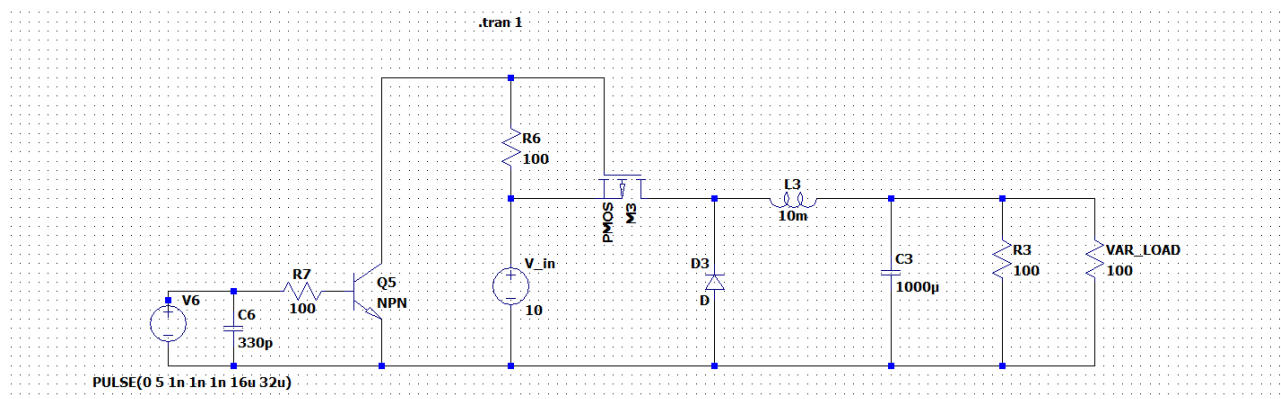


Figure 18: High-side buck converter configuration using PMOS.

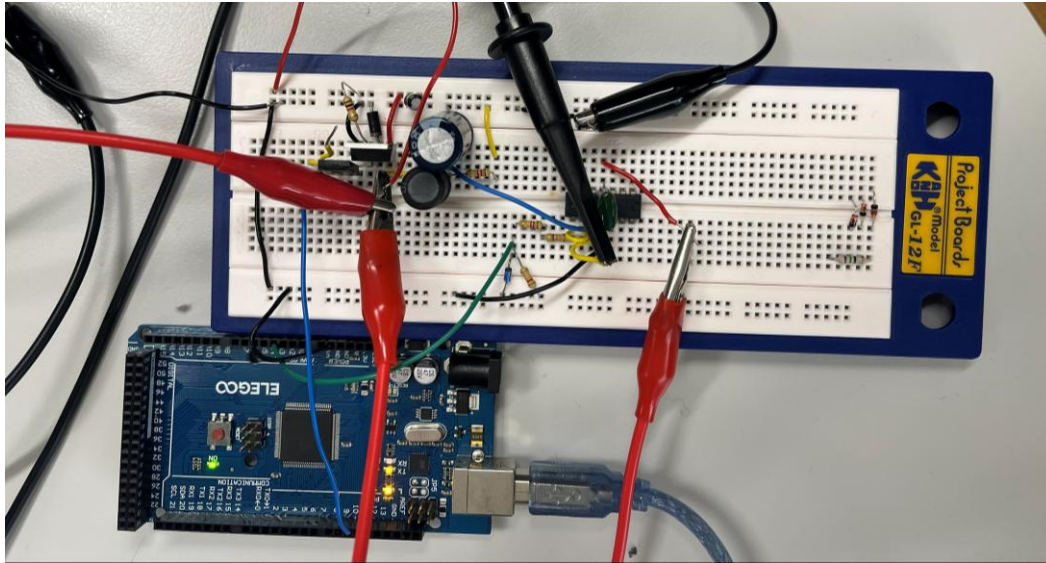


Figure 19: High-side buck converter configuration using PMOS breadboard.

Conduction mode

For all three iterations, a resistor was placed in parallel with the load. This was to prevent the buck converter powering the high-impedance load ($10\text{ k}\Omega$) from entering discontinuous conduction mode (DCM) and significantly lowering the output voltage. In DCM, since the diode only allows current to flow in one direction, the inductor current becomes saturated at zero for some time during each switching cycle in steady state. This occurs with high impedance loads which prevents the conduction of current across it, to an extent.

Results

Since the cascaded boost-buck converter was not successful connected to the Arduino together, this section will explain the results of just the buck converter.

The third iteration of the buck converter (buck converter with PMOS and gate driver) was connected to the voltage divider network, Arduino, and solar panel. Figure 20 shows how the buck converter was able to achieve a 5 V output ($\pm 0.3\text{ V}$) with a $1\text{ k}\Omega$ load resistor. Similarly, Figure 21 shows how it was able to achieve the same output with a $10\text{ k}\Omega$ load resistor. Furthermore, as seen in Figure 22, the settling time was around 6 seconds.

In summary, the requirement of a 5 V output of load 2 has been met for the full variation of load resistances, with successful connection to a PV panel and microcontroller. However, it does not meet the specification of a settling time within 4 seconds. To improve this, the P gains should be increased to an extent (the reason that the P gains were initially set as low was due to an experimentally-observed effect where the output was unstable, switching between 0 and 5 V – and that reducing the P gain seemed to help). Future work involves attempting to connect the buck and boost converter together and observing if the same results can be achieved.



Figure 20 - 5V DC output with 1 kΩ load resistor.



Figure 21 - 5V DC output with 10kΩ load resistor.

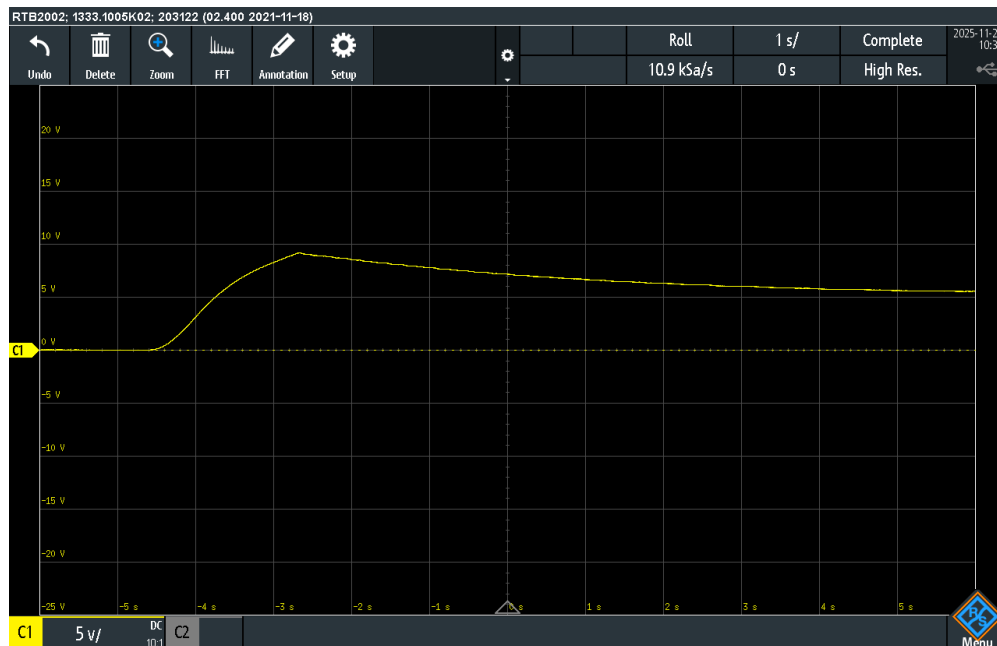


Figure 22 – Settling time of buck converter.

Software & hardware protection – N. Chen

From experimentation and testing, to collecting results, safety of humans and of equipment is of the utmost importance. As most components were rated for 30 V, a 25 V limit was applied at various nodes in the system, to protect the entire system from damage.

On the software side, binary logic was used to prevent any voltages over 25 V. The protection nodes included the output voltage of the panel (as this prevents high voltages flowing into the entire system), the output of the boost converter (as it is entirely possible that the voltage could be boosted above limits), and the output of the buck converter (in case there is a short-circuit, unaccounted for current surges, and to protect the load). The logic is as follows:

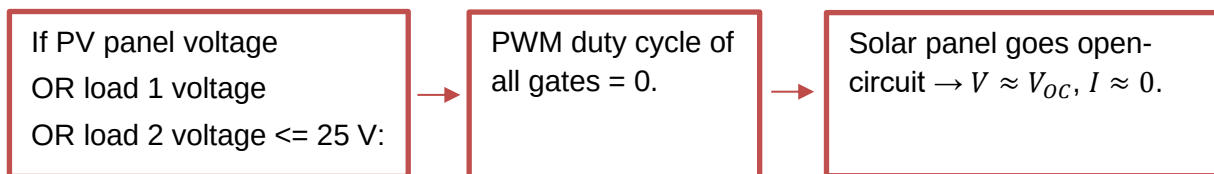


Figure 23 shows this logic in detail: if the voltage of any of the nodes listed above go beyond 25 V, an 'overvoltage protection' flag is set which is then multiplied by the output duty cycle. As seen in Figure 24, this means the duty ratio is set to 0 when the flag is 0.

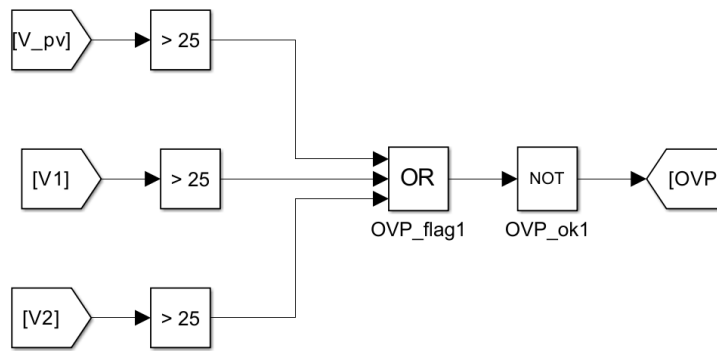


Figure 23: Software overvoltage protection – set flag.

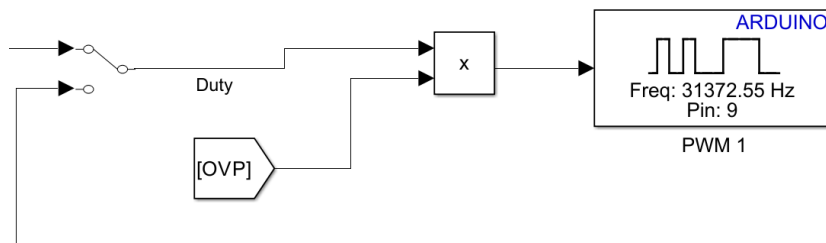


Figure 24: Software overvoltage protection – enact flag at gate.

If the software became redundant, protection is also available on the hardware side. Zener diodes (three 8.2 V Zener diodes in series) were placed at the output of the PV panel, load 1, and load 2. When the voltage at any of these nodes went above ~25 V, the Zener diode clamped the voltage, diverting excess current to ground. As an example, Figure 25 highlights the Zener diodes at the output of the PV panel and the output of load 1.

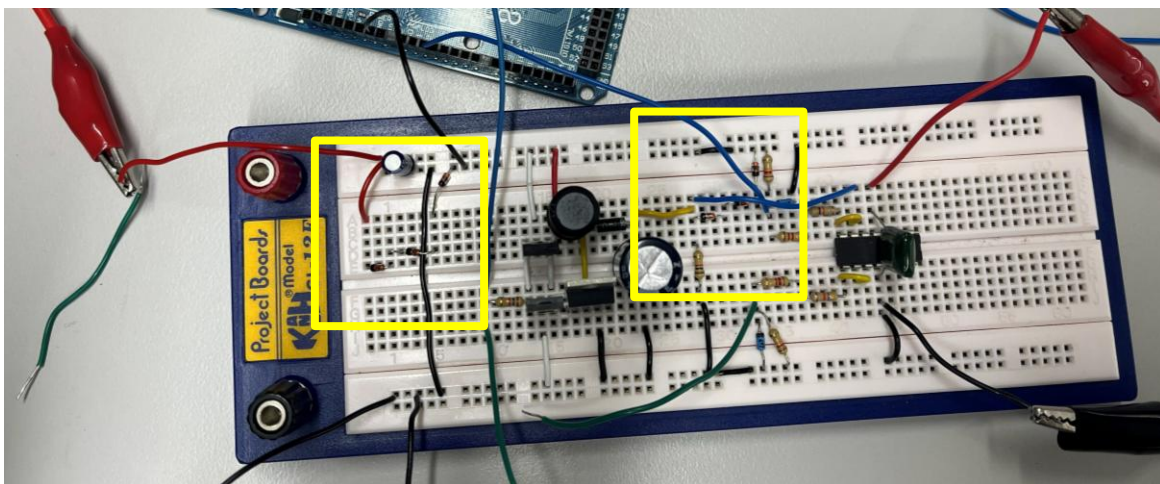


Figure 25: Zener diode overvoltage protection at output of PV panel and load 1 (yellow).

Gate driver design – N. Chen

Boost converter gate driver design

The N-channel MOSFET gate is a voltage-controlled element, controlling the flow of current from the drain to source via a gate voltage. This has many advantages such as a high-input impedance of the gate so that current isn't drawn from the microcontroller. However, it suffers from high gate capacitance as seen by the slow settling times in the level shifts of Figure 26. This affects the PWM, which affects the operation of the Boost converter.

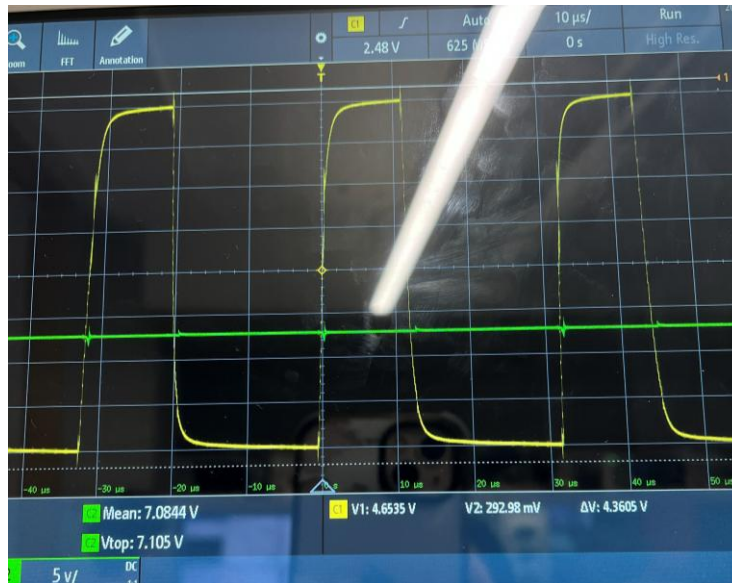


Figure 26: Boost - MOSFET gate without gate driver

This was fixed by using a totem-poll style driver to drive the gates – see Figure 27. Here, the pair of BJT transistors quickly charge or discharge the gate capacitance of the MOSFET by pushing current to charge the gate or pulling current to discharge the gate. The faster rise and fall times after the gate driver are seen in Figure 28.

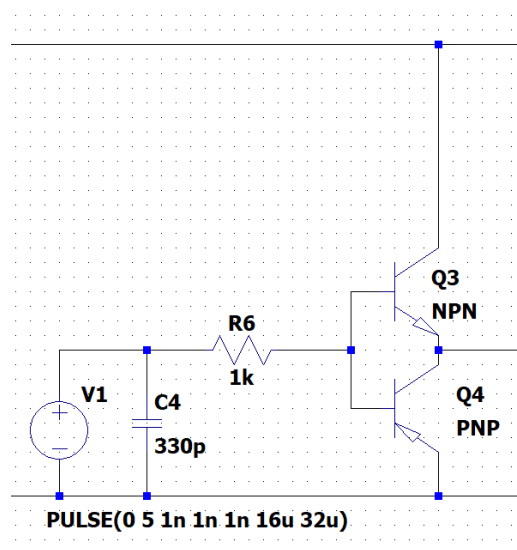


Figure 27: Boost gate driver SPICE



Figure 28: Boost - MOSFET gate driven with gate driver

Buck converter gate driver design

Similarly, the buck converter also required a gate driver. As seen in Figure 29, the P-channel MOSFET required a negative logic (i.e. $V_{GS} < V_{th}$) to turn on, and so a single NPN transistor was used to implement the logic signal. When the logic level was high, the MOSFET was conducting, and vice versa.

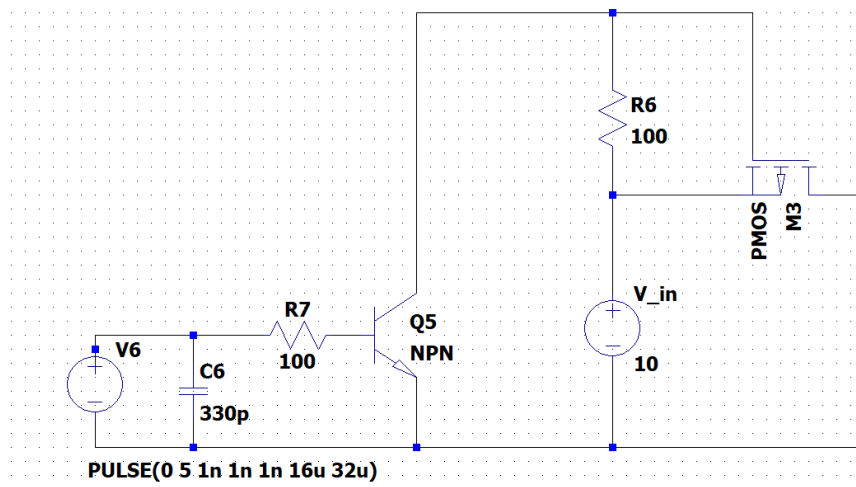


Figure 29: Buck gate driver SPICE

Simulink interface to Arduino design – N. Chen

The “Simulink Library for Arduino Hardware” was used to implement the control system on the Arduino MEGA 2560. As seen in Figure 30 and 31: the Simulink control blocks were edited such that all inputs came from the Analog input pins (pins A0 to A3), while the outputs went to the PWM output pins (pin 9 and 10).

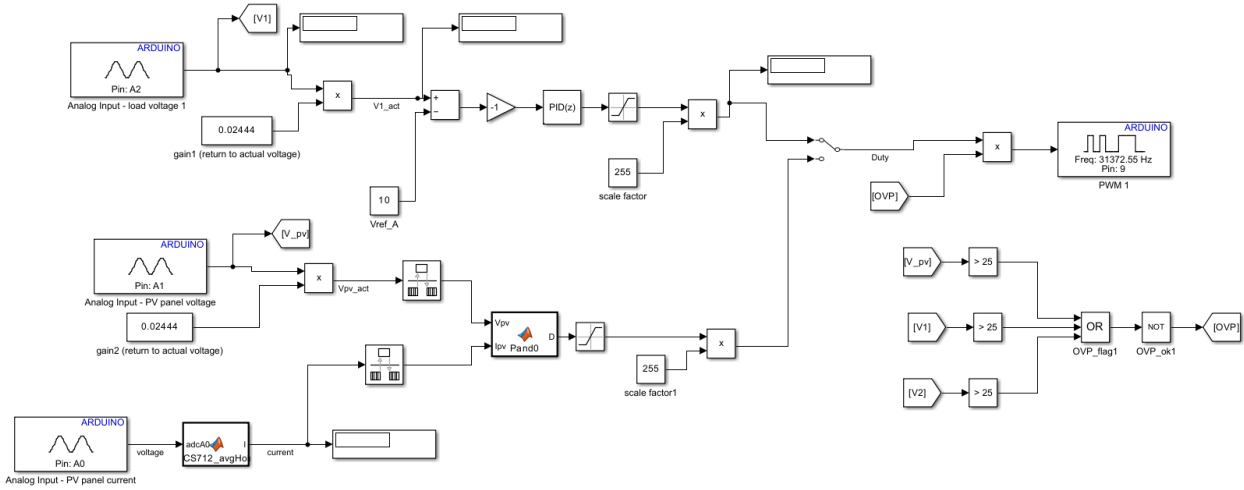


Figure 30: Simulink interface to Arduino for load 1

For the voltage inputs, since the microcontroller reads the raw analogue-to-digital (ADC) values, it needed to be converted back to the actual voltage reading. This was done by multiplying the ADC Analog input by a gain of:

$$V_{ref} \cdot LSB = \frac{V_{ref} \cdot VDM}{resolution - 1} = \frac{5 \cdot 5}{2^{10} - 1} = 0.0244$$

where VDM is the voltage division multiplier, and LSB is the Least Significant Bit. LSB is created through V_{ref} , the reference voltage set to 5 (V), which is the top of the microcontrollers analogue input range, and the $resolution$ of 2^{10} , which is set by the ADCs number of bits.

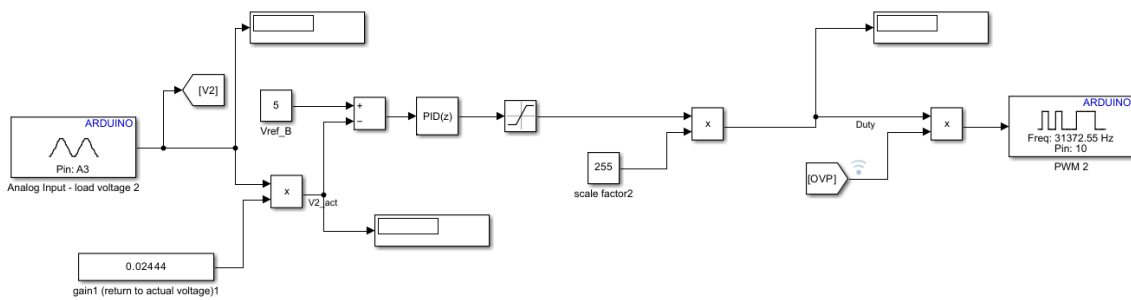


Figure 31: Simulink interface to Arduino for load 2

For the current input, the logic was similar to the voltage conversion, however a MATLAB function was used instead, as the logic was more complicated. It converted the raw ADC readings from the current sensor into current using 100-sample averaging. To get the current, the following logic was enacted:

$$I_{new} = \frac{avgADC \cdot (LSB - zeroPoint)}{scaleFactor} = \frac{avgADC \cdot (0.004888 - 2.488)}{0.185}$$

where *LSB*, is the Lease Significant Bit (same as earlier), *avgADC* is the average ADC reading from the current sensor over 100 samples, *zeroPoint* is the output voltage at 0 A (sensor offset) and *scaleFactor* is the sensitivity (V per A) for the current sensor (ACS712-5A). This code can be found in Appendix B.

Risk Mitigation Strategies

Table 3 lists some of the major technical and interpersonal risks associated with the project. In practice, topology failure was the only hazard that played out – the team’s response to this is documented in the Reflection.

Table 3 - Risk Assessment of risks pertaining to system design. The rubric used for analysis is from , and is presented in the appendix.

Risk	Probability	Severity	Rating	Mitigation Plan	Contingency Plan
Damaging components due to electrical shorts, exceeded power ratings, or reverse polarity connection	Likely	Moderate	Medium	Calculate current and power dissipation of components before turning supply on. Make sure power ratings are satisfied. Check ALL electrolytic capacitors before turning supply on. Plug in solar panel connection lead only when testing the circuit to prevent shorting the leads or accidental connections Check breadboard connections before turning any supply on	Purchase additional components in case of component damage. Clean up any breadboard designs to make layout cleaner and prevent any further issues.
Chosen converter topology not functioning as expected,	Likely	Significant	High	Research similar existing designs to verify viability.	Fall back on alternative topology options, such as changing from a

resulting in a non-functioning system				Simulate designs on Simulink and SPICE, to distinguish simulation artifacts. Ask mentors for advice on whether the design is viable.	cascaded design to a parallel design. As a worst-case scenario, design for only one load, or use a 5V linear regulator to obtain load 2.
Software error resulting in failure to connect or control Arduino via Simulink	Unlikely	Significant	Medium	Make sure all software is up to date, and that Arduino API match Simulink and MATLAB versions	Use Topic 2 template file to connect to Arduino, as this is known to work. As a worst-case scenario, use the Arduino IDE directly
Group conflict impeding progress	Unlikely	Moderate	Low	Make sure communication is clear and decision making is understood by everyone. Voice everyone's opinion when making major decisions.	Ask mentors to help mediate any major group conflict.

Reflection

Retrospectively, there are a few things that would change upon redoing this task. The first would be to complete the full task – connecting both converters together so that they work in tandem – however if our group knew how to do this successfully, we would have implemented this. So, in the future, one change would be to spend more time researching and analysing the problem so that we know which parts of our design are, or are not, an issue. I say this because, many times, we would rabbit-hole down a problem that we thought was a problem, which wasn't a problem. We ended up wasting time here. Asking for a second opinion about potential "problems" more often would have been helpful. Next time, we would also spend more time working with the solar panel and Arduino – we left that until last, which didn't give us enough time to solve the most important problems in this task. This would have been more helpful than spending about a week on working out how to reduce switching noise and ground-loop ringing.

Team dynamics worked well. The philosophy that we followed was that although we all had our separate tasks, it would do us good to have a solid understanding of the other team members tasks. Work was distributed as it came up. In retrospect, this could have been handled better –

i.e., planning for our tasks ahead more rigorously (however, it is hard to say if this is reasonable, since none of us had done a power electronics task to this complexity before). When one of us struggled to get a task done (a conflict), the others would help once they finished their own tasks. This was a successful way of working for us.

Many technical challenges have been mentioned in this report. Some of these include the reduction of noise, working out the implications of variable output from the PV panel, and getting the various systems to work cohesively. When these challenges came up, we would each go our own ways and research. This improved the chance that at least one of us would have a fix. For example, when the buck and boost converter did not work together, we came back to the lab with separate ideas of how to fix it – including putting the converters in parallel or working on the buck and boost separately until they could be put together.

Conclusion

Highly technical tasks require resilience, meticulousness to detail, discipline, and the ability to share information learned throughout the process. The task of building a power converter for a PV panel for two loads, controlled through MPPT and PI controllers through a microcontroller has resulted in the successful creation of a PV-panel-buck-converter-microcontroller system. Future work depends on successful operation of a PV-panel-boost-converter-microcontroller and its integration with the buck converter system. Overall, this task has been fruitful in allowing us to extend and challenge ourselves and each other.

Bibliography

- [1] Monolithic Power Systems, "Boost Converters (Step-Up Converter)," Monolithic Power Systems, [Online]. Available: https://www.monolithicpower.com/en/learning/mpscholar/power-electronics/dc-dc-converters/boost-converters?srsId=AfmBOopAVhQ2li0p671L6pPbdv8_IBEXpOHeMbhNS8sOS4x4sn44IE7Z. [Accessed 29 11 2025].
- [2] Mathworks, "MPPT Algorithm," Mathworks, 2025. [Online]. Available: <https://au.mathworks.com/discovery/mppt-algorithm.html>. [Accessed November 2025].
- [3] J. Falin, "Application Report SLVA255 - Minimizing Ringing at the Switch Node of a Boost Converter," Texas Instruments, 2006.
- [4] Microchip, "ATmega2560," 2025. [Online]. Available: <https://www.microchip.com/en-us/product/atmega2560#Documentation>. [Accessed 11 2025].
- [5] W. Xiao, Power Electronics Step-by-Step: Design, Modelling, Simulation, and Control, New York: McGraw-Hill Education, 2021.
- [6] D. Boers, "Beyond the risk matrix," ARMS Reliability, 2025. [Online]. Available: <https://www.armsreliability.com/page/resources/blog/beyond-the-risk-matrix>. [Accessed November 2025].
- [7] The Urban Scholar, "Effect of Variable Irradiance on a MPPT System on MATLAB Simulink," 26 October 2020. [Online]. Available: <https://howtoproject98.wixsite.com/theurbanscholar/post/effect-of-variable-irradiance-on-a-mppt-system-on-matlab-simulink>. [Accessed November 2025].

Appendix A

Risk Assessment Matrix

Risk assessment was conducted using the matrix shown in Figure 3.

		Impact →				
		Negligible	Minor	Moderate	Significant	Severe
Likelihood ↑	Very Likely	Low Med	Medium	Med Hi	High	High
	Likely	Low	Low Med	Medium	Med Hi	High
	Possible	Low	Low Med	Medium	Med Hi	Med Hi
	Unlikely	Low	Low Med	Low Med	Medium	Med Hi
	Very Unlikely	Low	Low	Low Med	Medium	Medium

Figure 3 - Risk assessment matrix used for

Table 3, from .

Appendix B

Current sensor reading code

The below code was used to convert ADC (A0) into current readings from the ASC712 current sensor:

```
function I = ACS712_avgHold(adcA0)

% Convert raw ADC readings from the current sensor (ACS712) into current
% using a 100-sample avg. Hold the prev. value between updates.

% Constants
vRef      = 5.0;           % ADC ref voltage (V)
LSB       = vRef / 1023.0; % Voltage per ADC count
zeroPoint = 2.488;        % Output voltage at 0 A (sensor offset)
scale_factor = 0.185;     % Sensitivity (V per A) for ACS712-5A
N         = 100;          % Samples per average window

% Persistent variables
persistent sum count I_prev
if isempty(sum)
```

```

        sum    = 0;
        count = 0;
        I_prev = 0;
end

% Accumulate the samples ---
sum    = sum    + double(adcA0);
count = count + 1;

% When window full, compute new average
if count >= N
    avgAdc = sum / N;
    Vout = avgAdc * LSB - zeroPoint;
    I_new = Vout / scale_factor;

    % Update the held current
    I_prev = I_new;

    % Reset accumulators
    sum = 0;
    count = 0;
end

% Output last valid current value
I = I_prev;
end

```